BD_HW8
۸۱۰۱۰۱۴۹۳
علیرضا کرمی

۱)

الف) خیر؛ اگر روی join یک شاخص داشته باشیم و قبل از انجام join، selection بزنیم، دیگر نمی‌توانیم از شاخص روی join استفاده کنیم چون عملا دیگر شاخصی روی آن مستقیم وجود نخواهد داشت. درصورتی با index داشتن روی join و انجام آن قبل selection، هزینه خیلی پایین می‌آید اما اگر نتوانیم مثل حالت گفته شده index بزنیم، هزینه خیلی بالا میره

ب) بله؛ زمانی مجبور به write می‌شویم که به آن حاصل sort شده یا حذف تکراری‌ها یا تاپل‌های sort شده نیاز داشته باشیم. اگر فرض کنیم که مثلا هر دو relation ما از قبل براساس attr اول join، مرتب شده‌اند (مثلا مثل سوال 6، مثلا B+tree به صورت cluster هست که یعنی sort شده است. پس خودش به صورت مرتب ذخیره شده) دیگر نیاز به مرتب‌سازی دوباره و ذخیره‌سازی در یک temp جدید نیست

ادامه ۱)

ح) تنها فرق external و internal در همان Pass 0 است. در external $\left\lceil \frac{N}{B} \right\rceil$ ران اولیه و در internal به اندازه $\left\lceil \frac{N}{2B} \right\rceil$ ران اولیه داریم. پس برای برابر شدن باید دو طرف فرمول زیر با هم برابر شوند:

$$2N\left(1+\left\lceil \log_{B-1}\left\lceil \frac{N}{B}\right\rceil \right\rceil\right) \overset{?}{=} 2N\left(1+\left\lceil \log_{B-1}\left\lceil \frac{N}{2B}\right\rceil \right\rceil\right)$$

مثلا $N=50$ و $B=10$، پس داریم:

$$2\times 50\left(1+\log_9\left\lceil \frac{50}{10}\right\rceil\right) \overset{?}{=} 2\times 50\left(1+\log_9\left\lceil \frac{50}{20}\right\rceil\right)$$

$$\Rightarrow \quad 2\times 50\left(1+\overset{1}{\cancel{\log_9 5}}\right) = 2\times 50\left(1+\overset{1}{\cancel{\log_9 3}}\right) \Longrightarrow$$ هزینه‌ها با هم برابر شد

ادامه ۱)

۱) کلا اطلاعاتی مثل همون کاتالوگ سیستم نگهداری می‌شود (attr ها با تایپ و relation مربوط و حجم‌شون) و مثلا به ازای هر index ، ساختار اون (hash یا B+tree) و attr هایی که روی اونا index تعریف شده نگهداری می‌شود. برای هر رابطه هم خب لازم است که نام رابطه و ساختارشو و نام و تایپ تمام attr ها (و حتی اطلاعاتی راجع به ظرفیتشون و حجمشون) و محدودیت‌های روی اونا relation رو ذخیره کنیم.

اطلاعات مربوط به view ها [illegible] و سطح authorization و سطح access

(۲

۱. از hash: چون همه مساوی هستن و همه هم استفاده شده اند.
از شاخص ترتیبی هم می‌توان استفاده کرد چون prefix آمده است

(خودم برای equality search، hash را بیشتر ترجیح میدهم)

و همه attr ها نیامده

۲. نه از hash می‌توان استفاده کرد (چون مساوی نیست و بازه‌ای است)
نه از B+tree می‌توان استفاده کرد (چون a نیامده و prefix نشده)

۳. از B+tree می‌توان استفاده کرد چون a و b و c، prefix هستن
از هش نمی‌توان استفاده کرد چون برابر همه مساوی نیست و همه attr ها نیامده

۴. برای بخش a>2 چون prefix است می‌توان از شاخص B+tree (ترتیبی) استفاده کرد
اما چون در داخل پرانتز OR داریم، باید کلش رو به نوعی بررسی کنیم
از hash هم به هیچ عنوان نمی‌شه استفاده کرد چون a>2 مساوی نیست و OR داریم که یعنی اسکن همه.

۳)

برای sid. 6 sailor < 50000 به صورت cluster : $\frac{50000}{100000} \times 500 = 250$ Pages ← RF

وقتی که از B+tree استفاده می‌کنیم

$4 + 250 = 254$ IO

↙ ارتفاع درخت

برای sid. 6 sailor = 50000 به صورت cluster و یا uncluster: چون فقط یک record به این شکل است (چون کلیده)

وقتی که از B+tree استفاده می‌کنیم ⟶ $4 + 1 = 5$ IO

↙ درصورتی که Alt(2) باشه. اگه Alt(1) باشه نمی‌خواد

برای sid. 6 sailor < 50000 به صورت uncluster :

وقتی که از B+tree استفاده می‌کنیم

$250 \times 80 + 4 = 20004$ IO

همونطور که می‌بینید، این از مقدار scan کل page که 500 تا page داریم، خیلی بیشتر است

برای sid. 6 sailor = 50000 به صورت uncluster یا cluster : چون sid کلیده فقط یدونه ازش داریم پس

وقتی از hash استفاده می‌کنیم $2 + 1 = 3$ IO ← برای زمانی که Alt(2) باشه، اگه Alt(1) باشه نمی‌خواد

↙ هزینه hash برای uncluster

برای حالت ها sid. sailor < 50000 وقتی از hash استفاده می‌کنیم، چه برای حالت cluster و چه حالت uncluster، چون range query است، باید کل صفحات رو اسکن کنیم و اصلا hash index به درد نمی‌خوره.

خلاصه

< 50000 :

- B+tree:
  - $(250+4) = 254$: cluster
  - $(20000+4) = 20004$: uncluster
- hash ← کلا به درد نمی‌خوره و همه صفحات رو باید اسکن کنیم

= 50000 :

- B+tree:
  - $(4+1) = 5$: cluster
  - $(4+1) = 5$: uncluster
- hash:
  - $(2+1) = 3$: cluster
  - $(2+1) = 3$: uncluster

برای Alt(2) اضافه شده

در Alt(1) نداریم این 1 رو

۴)

الف) ابتدا باید 10000 تا file را scan کنیم و بخوانیم. همراه آن همزمان با projection و گرفتن attr های لازم آن ها را sort هم می‌کنیم و می‌نویسیم. سپس run هایی از tuple های لازم sort شده خواهیم داشت و هزینه ایجاد آن در Pass 0 برابر است با:

$$\underbrace{10000}_{\text{کل scan}} + \underbrace{5000}_{} = 15000$$

نوشتن tuple های sort شده <title, ename> (پس در کل 5000 تا page داریم الان)

حال باید تعداد run ها را در Pass 0 حساب کنیم. اگر از روش external استفاده کنیم، B تا صفحه را می‌خوانیم و run هایی به طول B تولید می‌کنیم. تعداد run ها برابر خواهد بود با $\left\lceil \frac{N}{B} \right\rceil = \left\lceil \frac{5000}{10} \right\rceil$ که مساوی 500 تا run به طول B=10 تا صفحه است. پس داریم

$$\left\lceil \frac{N}{B} \right\rceil = \left\lceil \frac{5000}{10} \right\rceil = 500 \text{ run of } 10 \text{ pages}$$

(البته اگر روش ما internal sorting بود، در Pass 0 به اندازه $\left\lceil \frac{N}{2B} \right\rceil = \left\lceil \frac{5000}{20} \right\rceil = 250$ تا run داشتیم به طول 20 تا page)

ب)

$$\left\lceil \frac{500}{9} \right\rceil = 56 \text{ run of } 10\times9 \text{ pages} \longrightarrow \text{Pass } 1$$

(55 تا 9 تایی و یدونه 5 تایی)
10x9 — 10x5

$$\left\lceil \frac{56}{9} \right\rceil = 7 \text{ run of } 10\times9\times9 \text{ pages} \longrightarrow \text{Pass } 2$$

(6 تا 9 تایی و یدونه ۲ تایی)
10x9x9 — 10x9x2

$$\left\lceil \frac{7}{9} \right\rceil = 1 \text{ run of } 5000 \text{ pages} \longrightarrow \text{Pass } 3$$

(1 تا run 7 تایی)

(اگر روش ما internal بود باز هم ۳ تا Pass دیگر داشتیم چون داشتیم:

$$\left(\left\lceil \frac{250}{9} \right\rceil = \overset{\text{27 ران کامل}}{28}\right.$$ طول 20x9 (ران آخر 7x20)

$$\left\lceil \frac{28}{9} \right\rceil = \overset{\text{۳ تا run کامل}}{4}$$ طول 20x9x9 (ران آخر 20x9x1)

و $\left\lceil \frac{4}{9} \right\rceil = 1$ طول 5000)

پس برای pass ها یه نوعی از فرمول هزینه $2N\left(\left\lceil \log_{B-1} \left\lceil \frac{N}{B} \right\rceil \right\rceil + 1\right)$ استفاده کردیم. پس هزینه pass های 1 و 2 برابر است با 2×5000 که همان خواندن و نوشتن آن page هاست. برای pass آخر می‌توانیم با دقت بیشتر بررسی کنیم

ادامه ۴)

ب) در صورتی که pass آخر را بعد از merge کردن و حذف tuple های تکراری در حافظه disk بنویسیم دوباره (که طبق اون optimization یک چیز کاملا اضافی و بی فایده است) همان 2x5000 هزینه می گیرد اما از آنجایی که optimization داریم و همان موقع sort تکراری ها را حذف می کنیم، در واقع کار pass آخر در حالت عادی + هزینه حذف تکراری ها را در یک مرحله mix انجام بدیم و بنابراین کافی است که همان لحظه sort، تکراری ها را حذف کنیم و همان را خروجی دهیم. پس کل هزینه pass آخر با optimization به 5000 کاهش می یابد. پس در کل داریم:

scan خواندن — نوشتن sort تاپل ها — توضیحات در صفحه بعد ***

$$\underbrace{10000 + 5000}_{Pass\ 0} + \underbrace{2\times5000}_{Pass\ 1} + \underbrace{2\times5000}_{Pass\ 2} + \underbrace{5000}_{Pass\ 3} = \boxed{40000}$$

(بهینه سازی باعث شد که یک خواندن و نوشتن در ابتدای کار کمتر داشته باشیم و یک خواندن و نوشتن در انتهای کار کمتر داشته باشیم. بدون بهینه سازی هزینه برابر بود با:

$$\left(\underbrace{(10000+5000)}_{\text{جدا کردن tuple های لازم}} + (4\times2\times5000) + \underbrace{5000}_{\text{کم گرفتن خروجی sort شده و حذف تکراری ها}} = 60000\right.$$

3x2x5000 برای pass ها اضافه می شود در واقع

ج) اگر clustered باشه خوبه و با یک هزینه scan اون N تا Page می تونیم به صورت sort شده داشته باشیمشون. اگر unclustered باشه اصلا خوب نیست چون باید به ازای تک تک record ID ها در page متفاوتی برویم که هزینه IO می شه: تعداد صفحه × تعداد recordID در هر صفحه یعنی به تعداد tuple ها و بیشتر از تعداد page های کل، باید IO بزنیم. که اگر کلش را scan می کردیم، به مراتب هزینه اش کمتر می شد. البته البته البته اگر همان (ename,title) در rid ما در unclustered باشد چون نیاز به بقیه attr ها نخواهیم داشت و rid ها sort شده هستند پس چون IO اضافی نمی زنیم که کل attr ها را داشته باشیم، پس sort در page های برگ حاوی rid برای ما کافی است پس unclustered هم همچون clustered (که فقط یک scan کل داره) برای ما خوب خواهد بود.

ادا- ۴) ب) اگر حالت بسیار ساده و عادی را در نظر بگیریم، برای هر ۳ تا pass اضافه در مجموع به اندازه 5000 x 2 x 3 تا IO هزینه می دهیم اما اگر عمیق تر نگاه کنیم، با اینکه 5000 x 2 x 3 طبق یکی از پاسخنامه‌های پارسال است اما با توجه به مطالبی که استاد در کلاس درس دادن و شخصا پرسیدیم از ایشون و توی صفحه قبل هم توضیح دادم، برای pass آخر، همان لحظه ای که merge می کنیم، تکراری ها را حذف می کنیم و چون به جواب رسیدیم، لزومی ندارد که جواب را در disk بنویسیم. درصورتی که بنویسیم، همان 5000 x 2 برای pass سوم هزینه شده اما اگر نوشتن آخر را در نظر نگیریم (که منطقا نباید در نظر گرفت) هزینه برابر 5000 خواهد بود که همان است که آوردم

چرا که دوباره باید 5000 تا بخواند

(می دانم در پاسخنامه‌تان برای 3 pass هم همان 5000 x 2 را گرفتید ولی جواب من درسته)

(5

الف) از آنجایی که B=3 ، برای استفاده از hash join در برای اینکه recursive partitioning نداشته باشیم ، باید داشته باشیم $\left\lceil \frac{Min(M,N)}{B-1} \right\rceil < B-1$ یعنی باید $\left\lceil \frac{50}{2} \right\rceil < 2$ باشد که برقرار نیست. بنابراین hash join ساده نمی‌توان استفاده کرد. از آنجایی هم که index نداریم ، از INLJ نمی‌توانیم استفاده کنیم. پس برای روش‌های دیگری داریم :

Page Oriented Nested loops Join: $(M + M\times N)$

$$\begin{cases} 50 + (50\times 1200) = 60050 \\ 1200 + (1200 + 50) = 61200 \end{cases}$$

Block Nested loops join: $M + \left\lceil \frac{M}{B-2} \right\rceil \times N$

$$\begin{cases} 1200 + \left\lceil \frac{1200}{1} \right\rceil \times 50 = 61200 \\ 50 + \left\lceil \frac{50}{1} \right\rceil \times 1200 = 60050 \end{cases}$$

Sort merge Join

از روش internal استفاده می‌کنیم تا هزینه کمتر و بهتر شود

$$\underbrace{2M\left(1+\left\lceil \log_{B-1}\left\lceil \frac{M}{2B} \right\rceil \right\rceil\right)}_{\text{M هزینه sort اول}} + \underbrace{2N\left(1+\left\lceil \log_{B-1}\left\lceil \frac{N}{2B} \right\rceil \right\rceil\right)}_{\text{N هزینه sort اول}} + \underbrace{(M+N)}_{\text{هزینه merge}}$$

$$\Longrightarrow 2\times 1200\left(1+\left\lceil \overbrace{\log_2 200}^{8} \right\rceil\right) + 2\times 50\left(1+\left\lceil \overbrace{\log_2 9}^{4} \right\rceil\right) + 1200+50$$

$$= 2\times 1200\times 9 + 2\times 50\times 5 + 1200+50$$

$$21600 + 500 + 1250 = 23350$$

(اگر از external استفاده می‌کردیم باید در ابتدا با $\left\lceil \frac{N}{B} \right\rceil$ یا $\left\lceil \frac{M}{B} \right\rceil$ ران شروع می‌کردیم و داشتیم :

هزینه merge sort است) $\left(2\times 1200\times 10 + 2\times 50\times 6 + 1200+50 = 24000 + 600 + 1250 = 25850 \right.$

(کم‌ترین internal شد و منطقاً باید بتونیم توضیح ولی جنبه‌ای مسئله که چون بافر کمه ، internal نمیشه رفتیم external ولی باز

$\Longrightarrow$ پس کم‌ترین هزینه برای internal merge sort است با هزینه 23350 . در رتبه دوم external merge sort

پس کلاً merge sort در این رابطه الف ، کم‌ترین هزینه را دارد.

ادامه 5)

ب) از آنجایی که داریم:
(B=11)

$$\left\lceil \frac{\min(M,N)}{B-1} \right\rceil = \frac{50}{10} = 5 < 10$$

⟸ پس در این حالت می‌توانیم از hash join استفاده بکنیم و داریم:

hash join هزینه: $2\times 1200 + 2\times 50 + (1200+50) = 3(1200+50) = \underline{3750}$

برای بقیه هم داریم:

$$\text{PONLJ}: \begin{cases} 50 + 1200\times 50 = 60050 \\ 1200 + 50\times 1200 = 61200 \end{cases}$$

$$\text{Block nested LJ}: \begin{cases} 50 + \left\lceil \frac{1200}{9} \right\rceil \times 50 = 50 + 134\times 50 = 6750 \\ 1200 + \left\lceil \frac{50}{9} \right\rceil \times 1200 = 1200 + 6\times 1200 = 8400 \end{cases}$$

$$\text{Internal Sort Merge J}: 2\times 50\times\left(1+\log_{10}\overset{\lceil \frac{50}{22}\rceil}{3}\right) + 2\times 1200\left(1+\log_{10}\overset{\lceil \frac{1200}{22}\rceil}{54}\right) + \overset{\text{merge}}{1250}$$

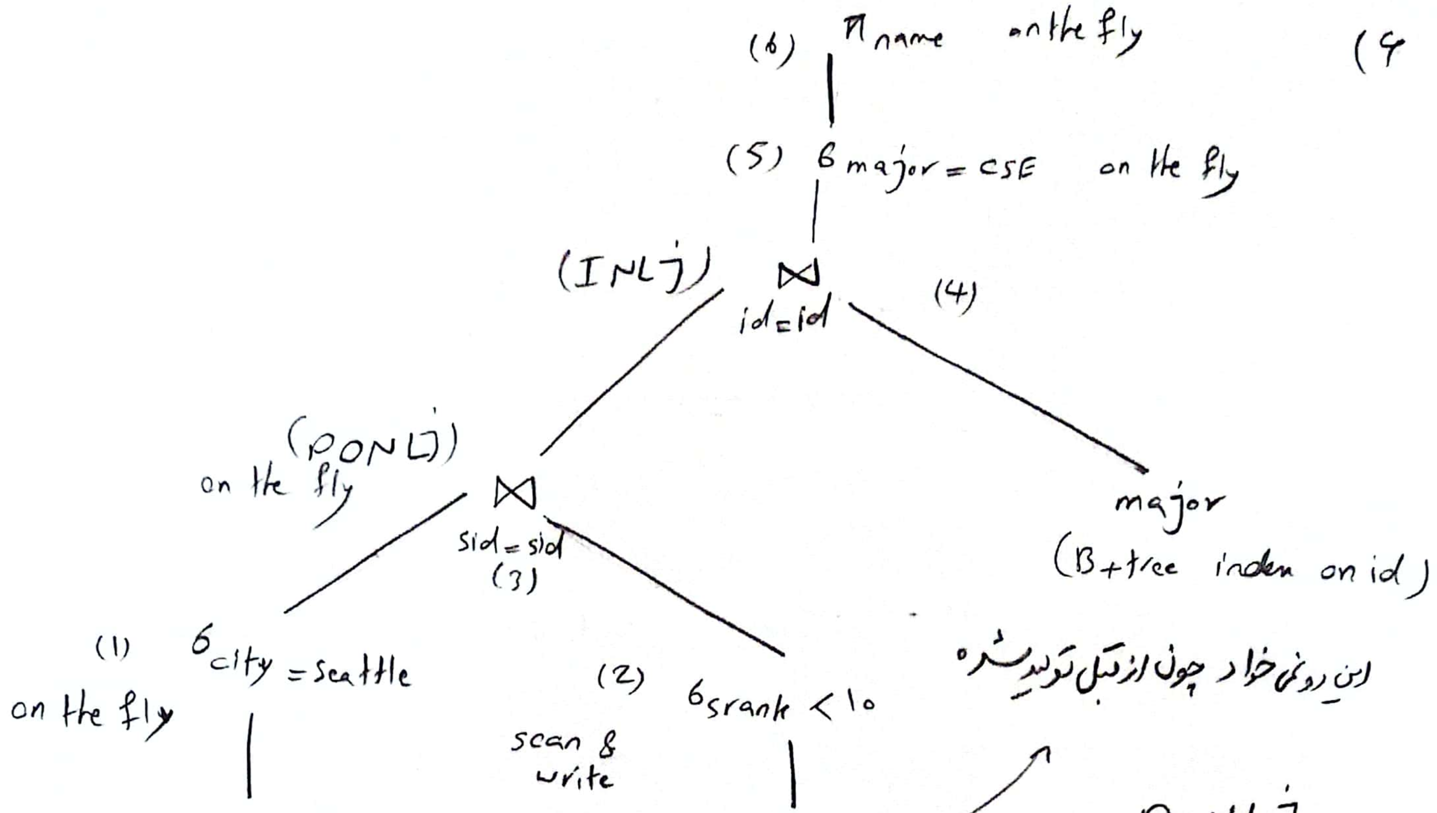
$$= 100\times 2 + 2400\times 3 + 1250 = 8650$$

$$\text{External Sort Merge J}: 2\times 50\times\left(1+\log_{10}5\right) + 2\times 1200\left(1+\log_{10}110\right) + 1250$$

که بیشتر از حالت internal می‌شود.

⟸ پس بین تمام حالات و روش‌های مختلف، بهترین روش همان hash join ساده است با IO 3750

(4


(6) π name on the fly
(5) σ major = CSE on the fly
(INLJ) ⋈ id=id
(4)
(PONLJ) on the fly ⋈ sid=sid (3)
major (B+tree index on id)
(1) σ city = Seattle
on the fly
(2) σ srank < 10
scan & write
Applicants
School

این رو نمی خواد چون از قبل تولید شده

$$\underset{\text{scan}}{100} + \underbrace{10 + \cancel{\alpha}}_{\text{scan \& write برای school}} + \overbrace{\overset{\text{page تا 5}}{\cancel{\left(100\times\frac{1}{20}\right)}} + \overset{\text{page } 5\times 1}{\left(100\times\frac{1}{20}\right)\left(\left\lceil 10\times\frac{9}{100}\right\rceil\right)}}^{PONLJ} + \overbrace{\left(\cancel{\alpha} + 1\times 9\times(3+1)\right)}^{INLJ} = \underbrace{100+10+5+36}$$

این رو نمی خواد چون on the fly همش رو انجام داد

این رو نمی خواد چون on the fly است

تعداد کل IO های لازم برابر است با ← **151** ⟸

نکته (inner باید به صورت scan & write باشد در مرحله (2) اما outter در مرحله (1) ، on the fly باشد)

برای INLJ : $M + M \times P_r \times \text{lookup}$

M ← تعداد page های outter

P_r ← record per page

lookup → 3 + 1 ← چون id کلید است پس حداکثر یک رکورد داریم ازش و با اینکه unclustered است ، فقط با چک کردن 1 page کافی است .

برای PONLJ : $R + R \times S$

R ← outter

S ← inner

$100 \times \frac{1}{20} = 5$ ← seattle یکی از 20 شهر است پس

page های Applicant با Seattle

srank<10 → $\frac{9-1+1}{100-1+1} = \frac{9}{100}$ ← 9 تا تاپل از 10×10 تا تاپل school که در 1 page جا میگیره